Neural Networks – Part 3

Machine Learning and Big Data - DATA622

CUNY School of Professional Studies

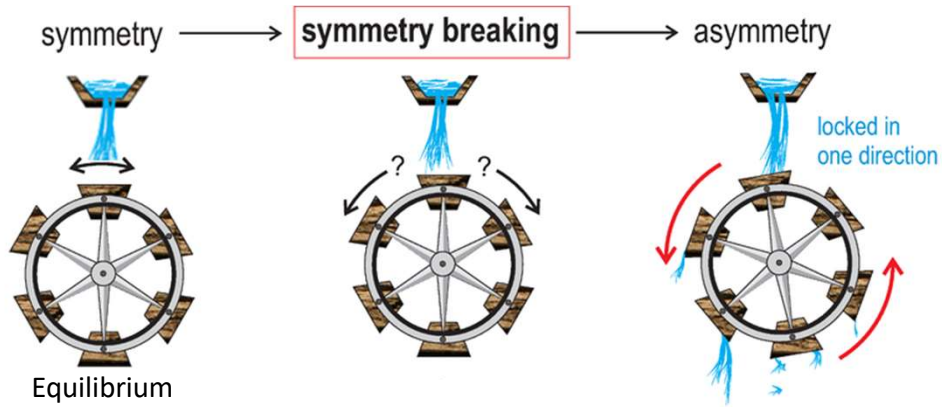
Lottery ticket hypothesis

Breaking symmetry of neural networks with random initialization

CUNY School of Professional Studies

Breaking symmetry

- Initializing weights & biases to zeroes (or ones) is problematic

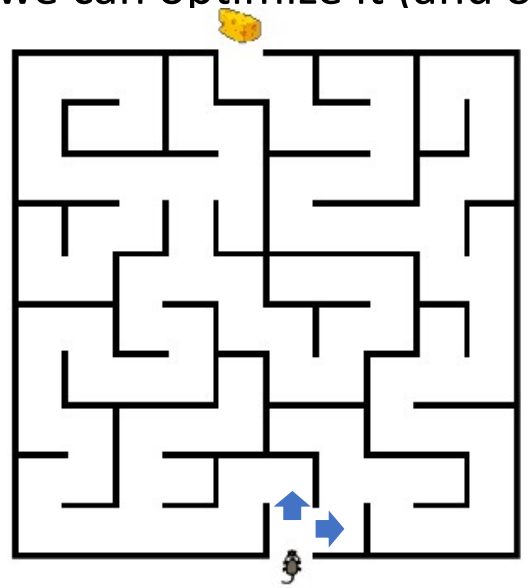


- If all weights are zeros, subsequent hidden layers will get zero signal
- If weights are all the same value (e.g. 1), then subsequent hidden layers will get exactly the same signal i.e. if all weights are initialized to 1, each unit gets signal equal to sum of inputs (and outputs sigmoid(sum(inputs))).
- No matter what was the input - if all weights are the same, all units in hidden layer will be the same too. This is why you should initialize weights randomly.

Mouse Maze

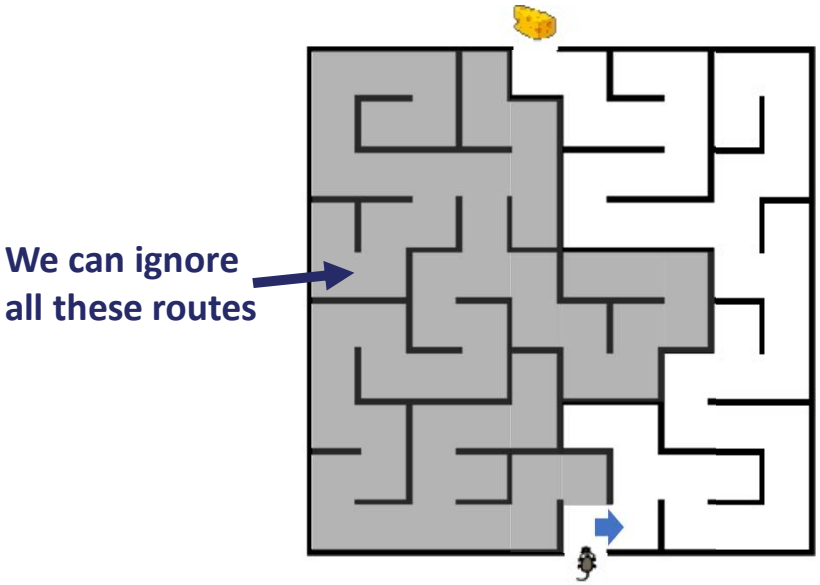
Consider a mouse trying to find a way through a maze:

1. The initial routing will be critical in terms of how quickly a solution is found
2. Once a route is found, we can optimize it (and other routes are ignored)



Mouse Maze

If we have a favorable start and found a route, then the rest of the maze is irrelevant



Lottery ticket hypothesis

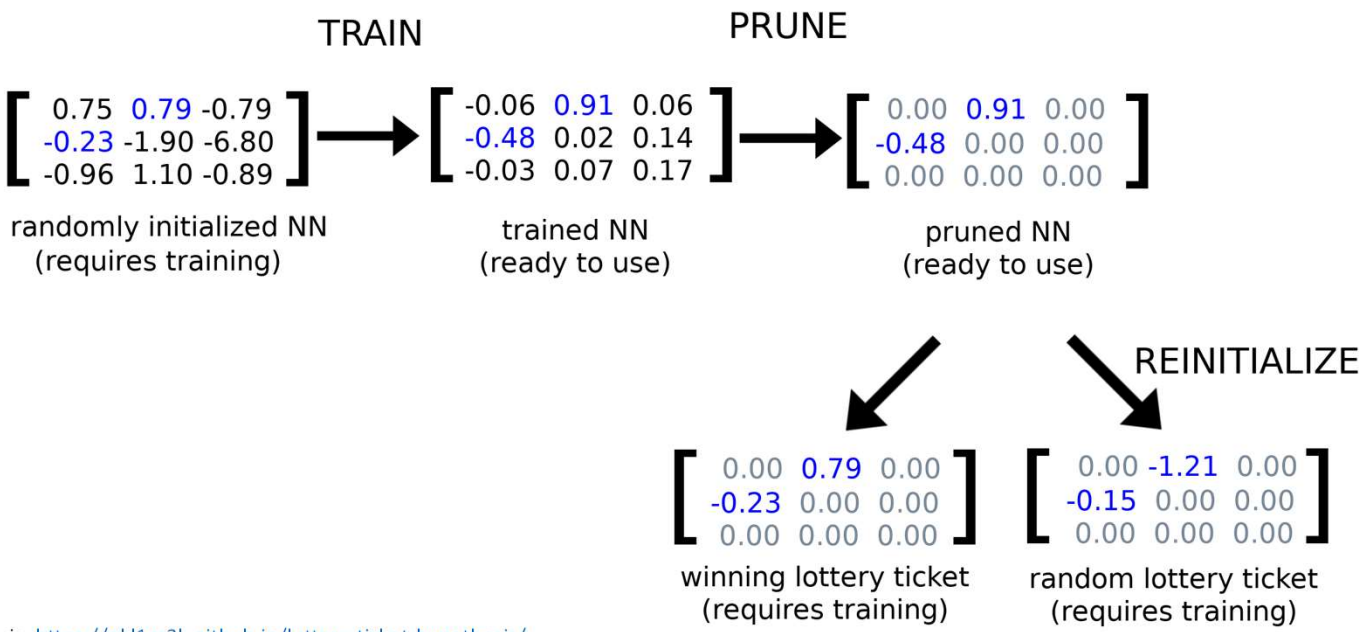
"A randomly-initialized, dense neural network contains a subnetwork that is initialized such that-when trained in isolation-it can match the test accuracy of the original network after training for at most the same number of iterations."

The subnetwork is referred to as a winning ticket (it has won the initialization lottery!)

Lottery ticket hypothesis: https://arxiv.org/pdf/1803.03635.pdf

Lottery ticket hypothesis

- A neural network is typically sparse (most of the weights are zero)
- Pruning the network doesn't change accuracy but makes the model faster
- The lottery ticket (initial optimal weights) will regenerate the optimal model



Lottery ticket hypothesis: https://okl1m3k.github.io/lottery-ticket-hypothesis/

Why do you need so many layers?

Feature Representation

CUNY School of Professional Studies

How many layers do we really need?

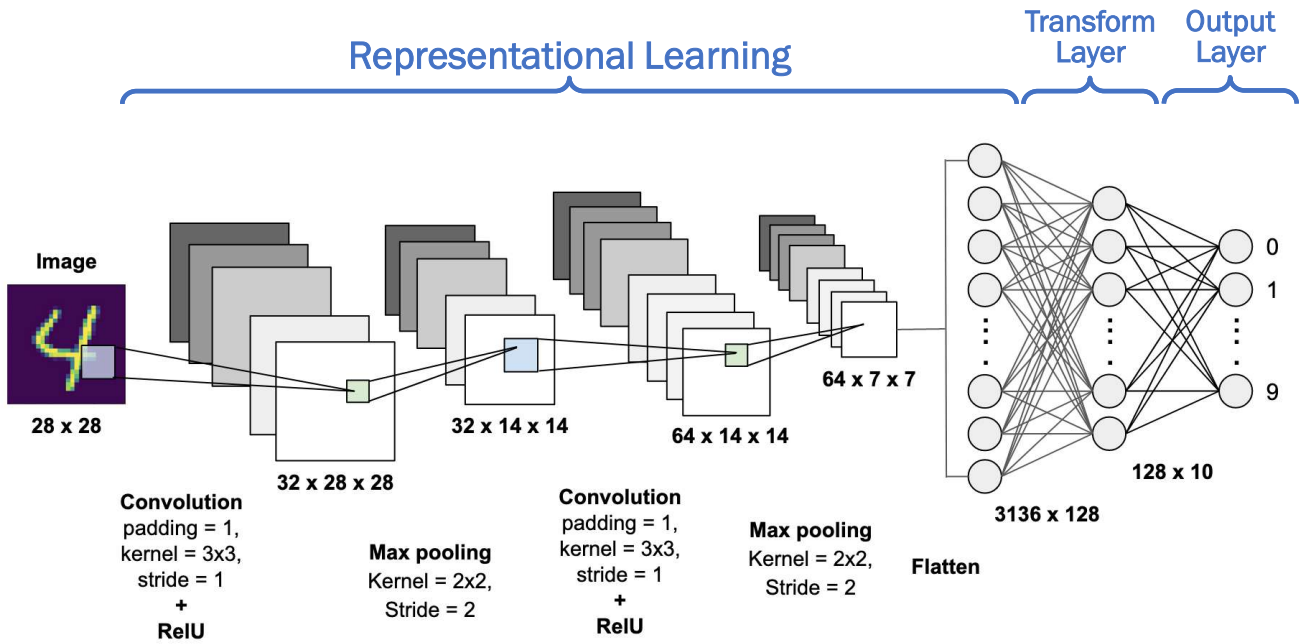
The Universal Approximation Theorem tells us:

- A neural network with only 1 hidden layer can approximate any continuous function
- Accuracy of approximation can be controlled by number of layers & neurons

Num Hidden Layers	Result
none	Only capable of representing linear separable functions or decisions.
1	Can approximate any function that contains a continuous mapping from one finite space to another.
2	Can represent an arbitrary decision boundary to arbitrary accuracy with rational activation functions and can approximate any smooth mapping to any accuracy.
>2	Additional layers can learn complex representations (sort of automatic feature engineering) for layer layers.

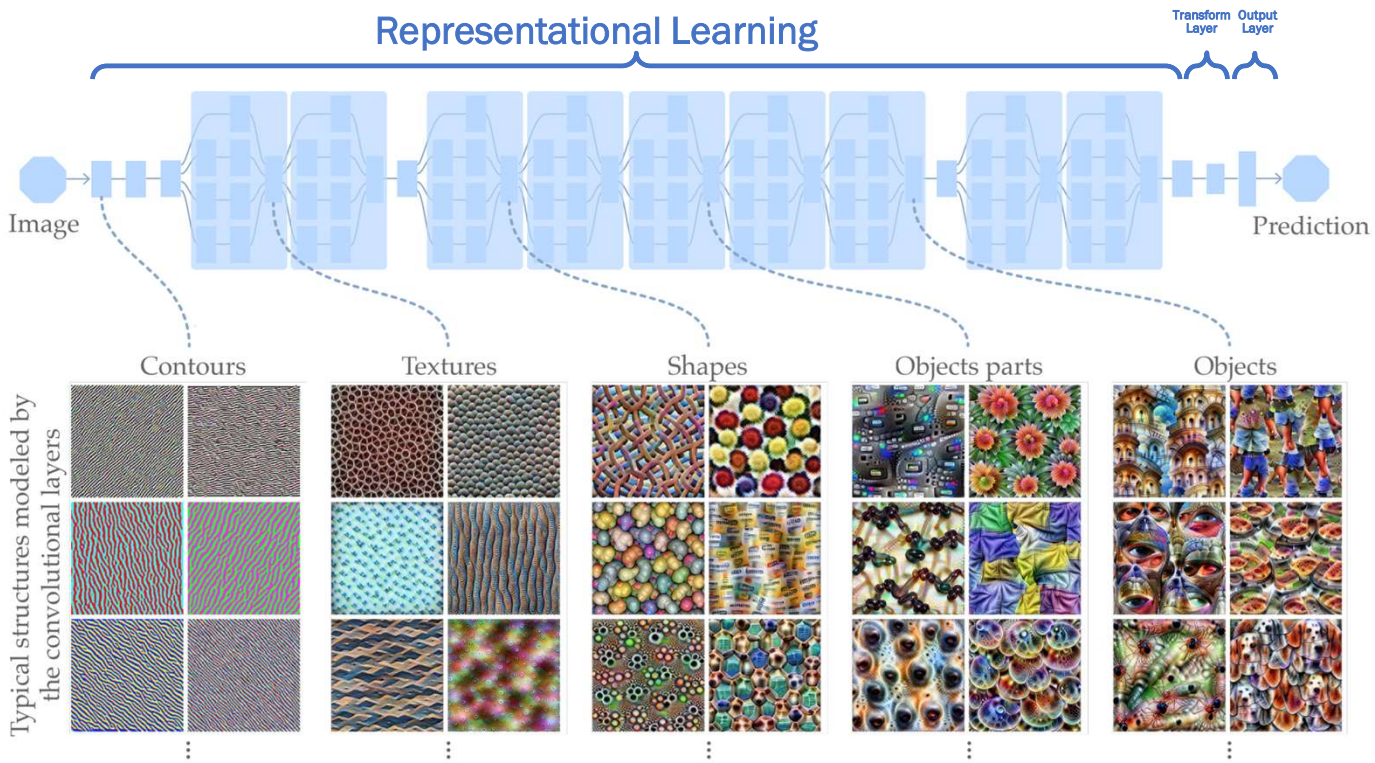
So why do Neural Network (Deep Learning) have so many layers?

Feature Representation



ConvNet=Convolutional Neural Network, used for images.

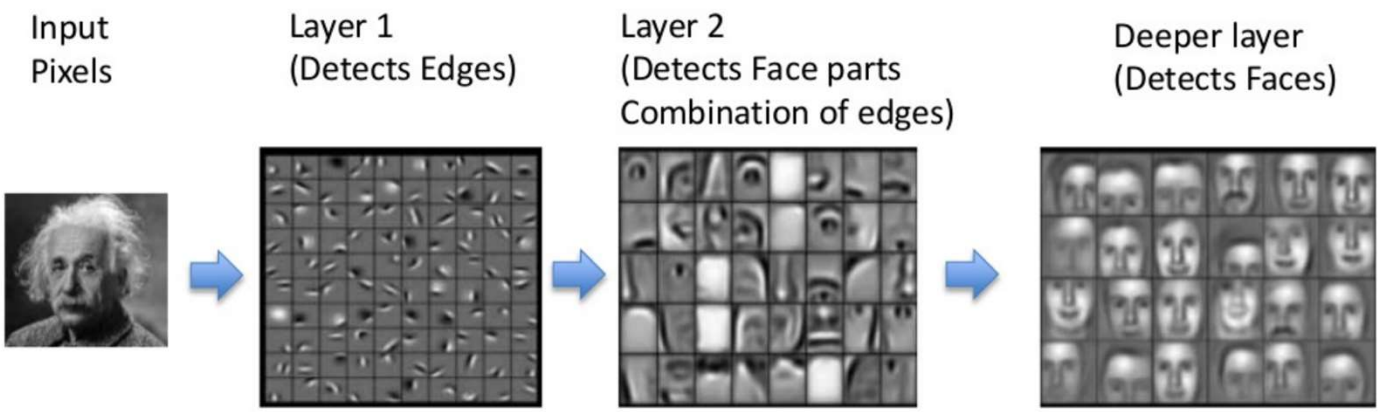
Representational Learning

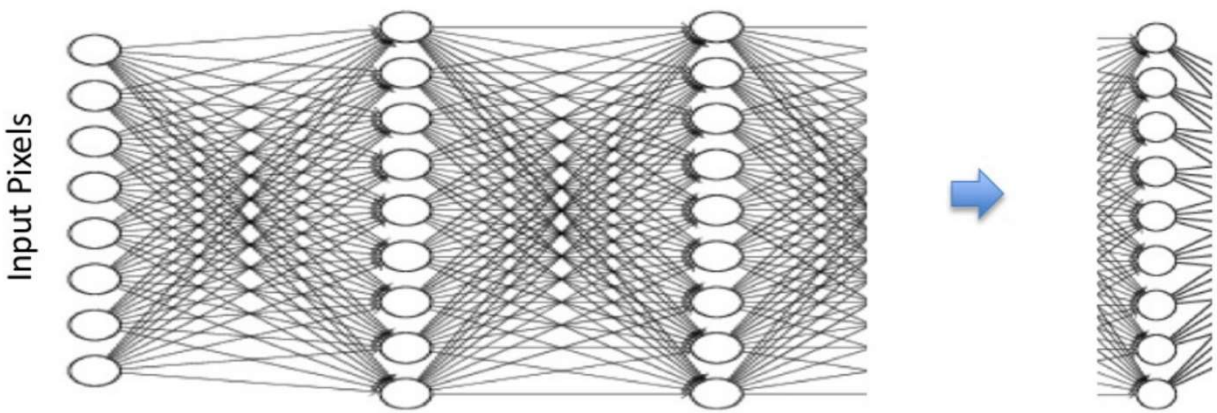


Successive model layers learn deeper intermediate representations

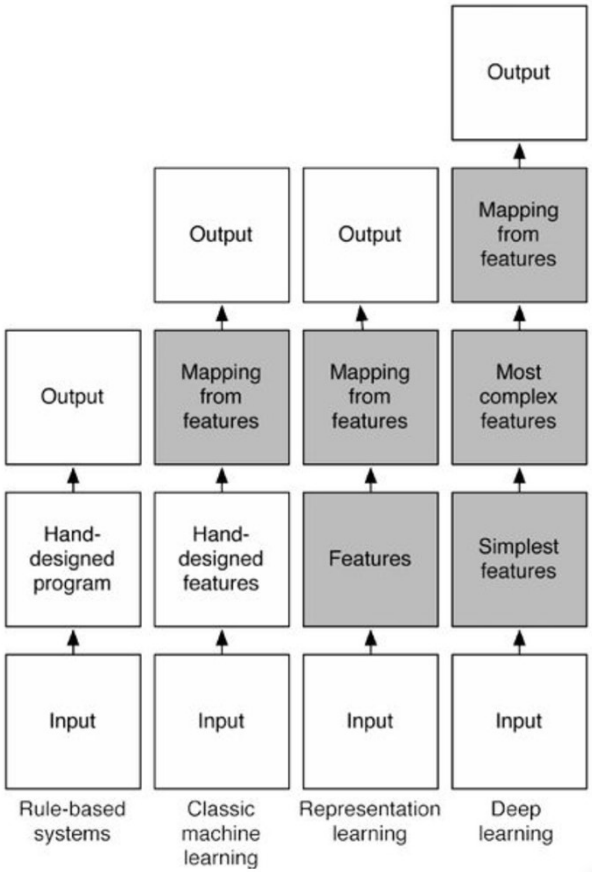
Source: TLL
Source: Improving Latent Representations of ConvNets for Visual Understanding, Thomas Robert

Representational Learning





Representational Learning



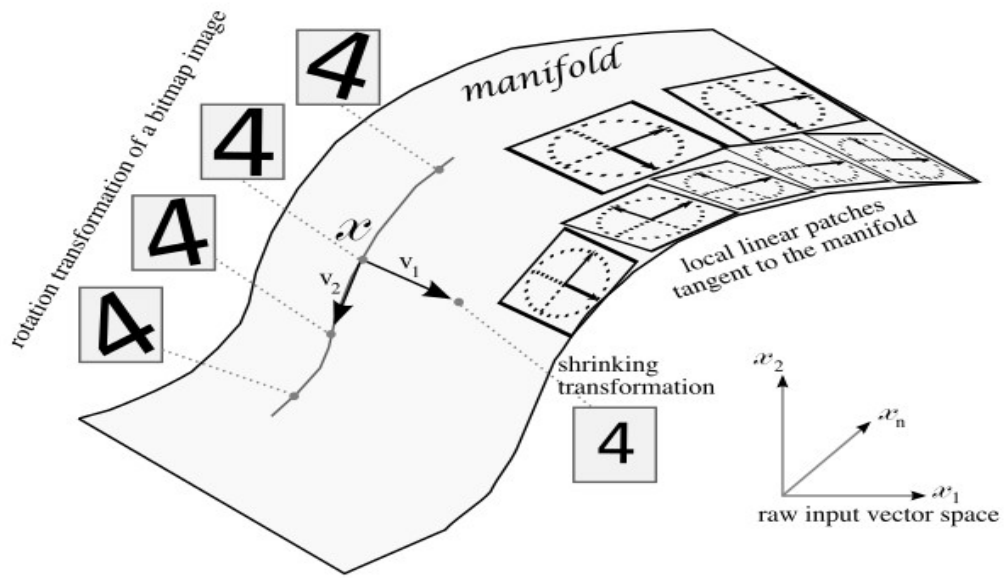
Manifold Hypothesis

Solution space of data

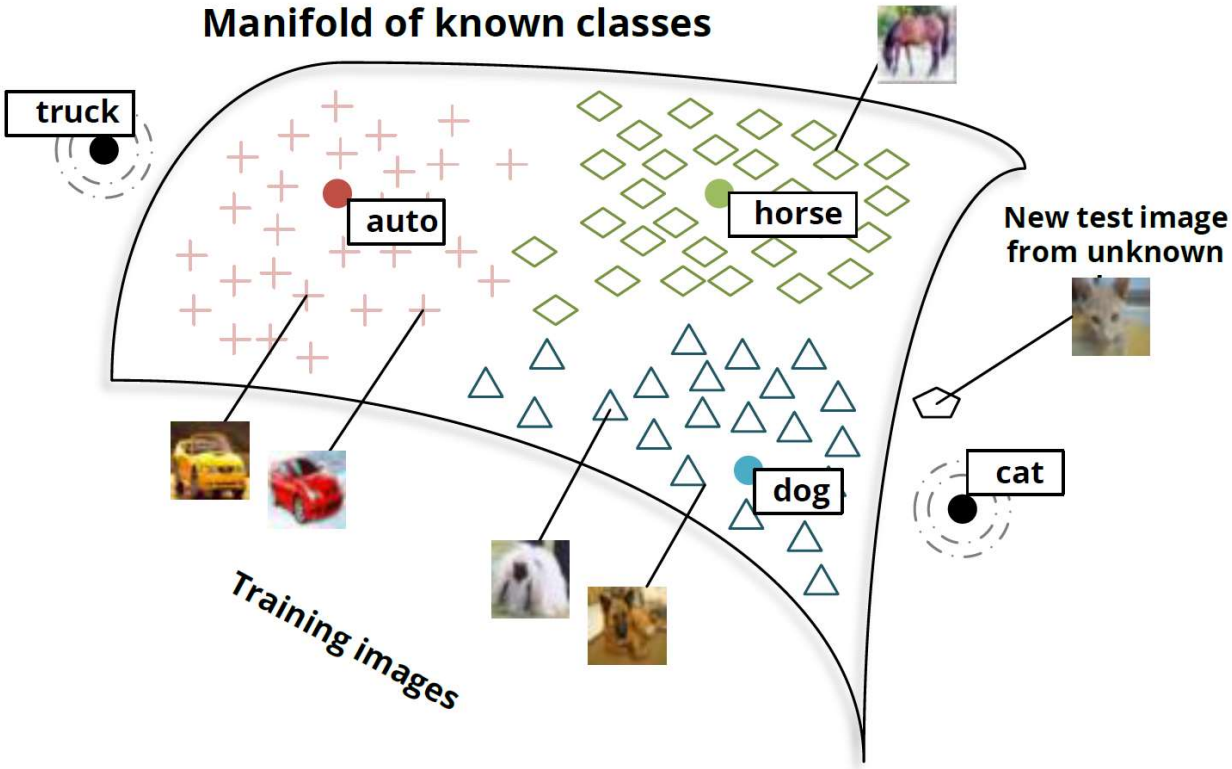
CUNY School of Professional Studies

Manifold Hypothesis

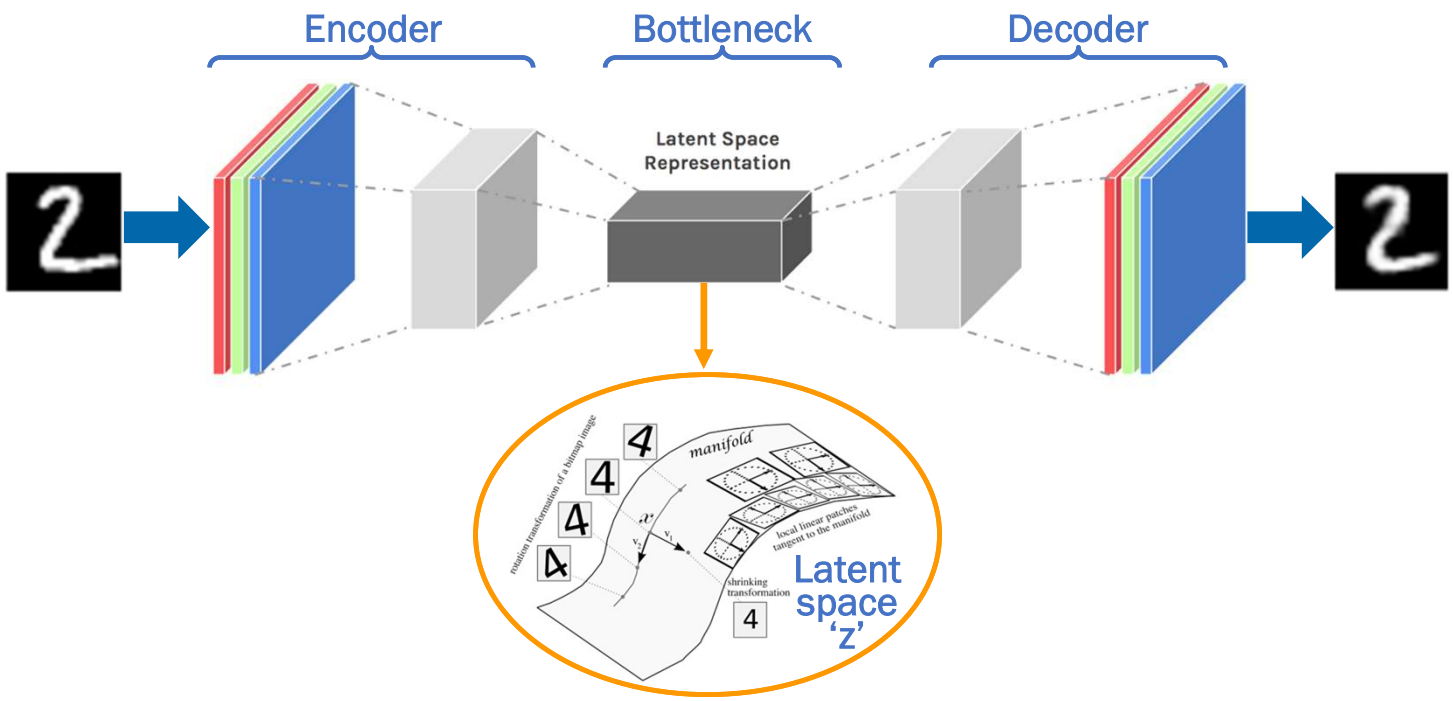
- Manifold Hypothesis states that natural data forms lower dimensional manifolds in its embedding space.
- There are both theoretical and experimental reasons to suspect that the Manifold Hypothesis is true



Another view of manifolds

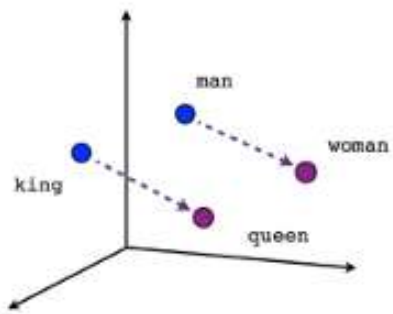


Auto-encoders

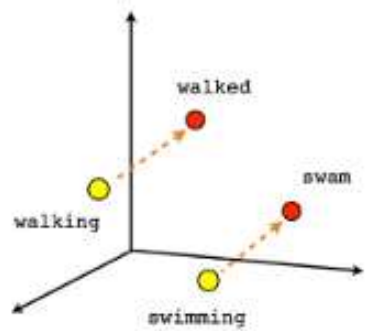


Source: Hackernoon Latent Space Visualization

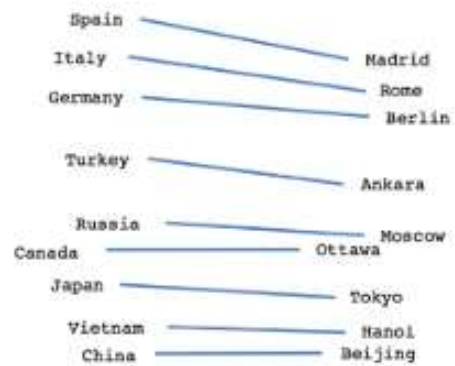
Latent-space vectors



Male-Female

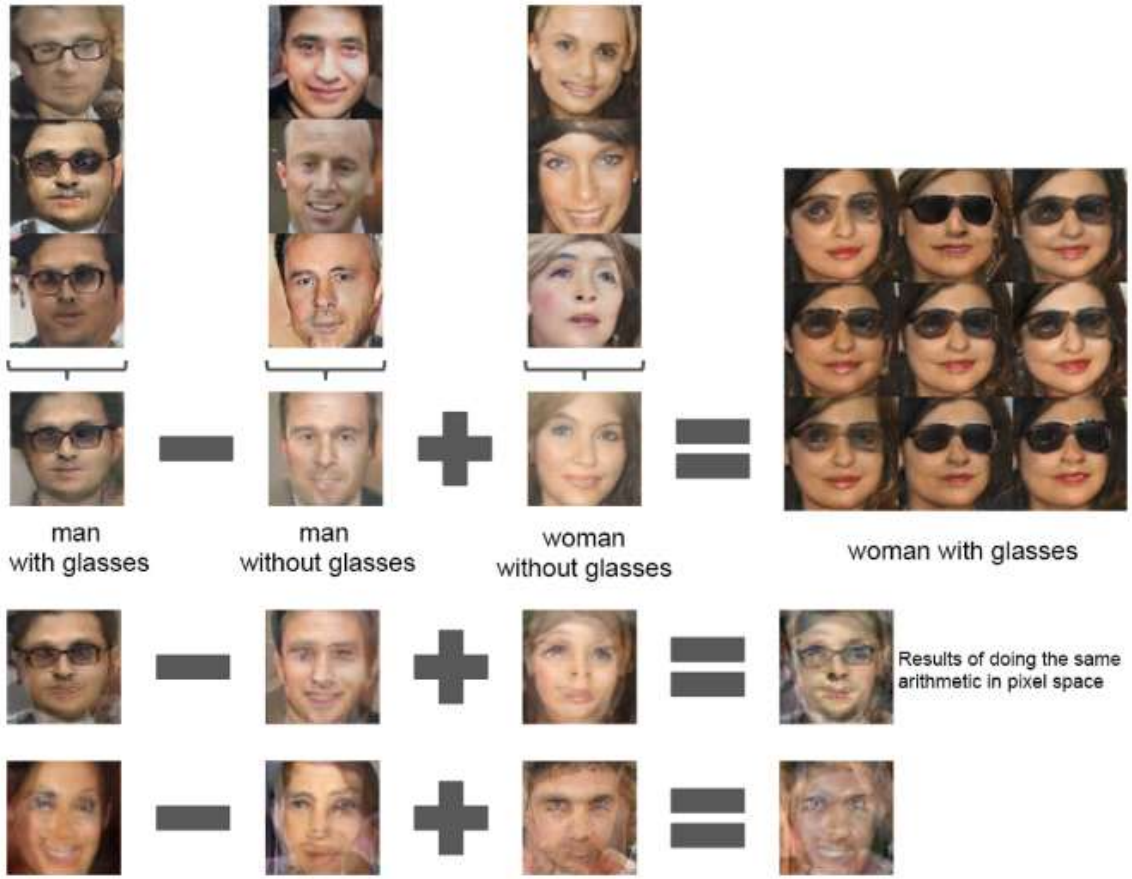


Verb tense



Country-Capital

Arithmetic with latest space vectors



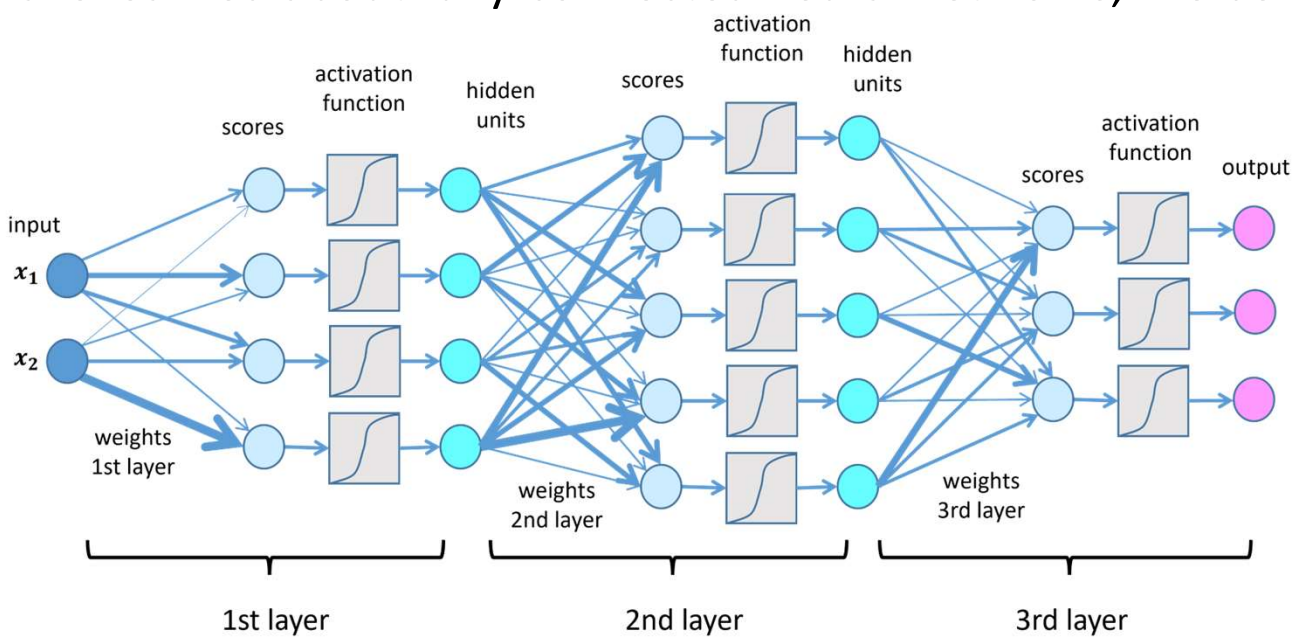
Convolutional Neural Networks

Image processing (inspired by nature)

CUNY School of Professional Studies

Image Models

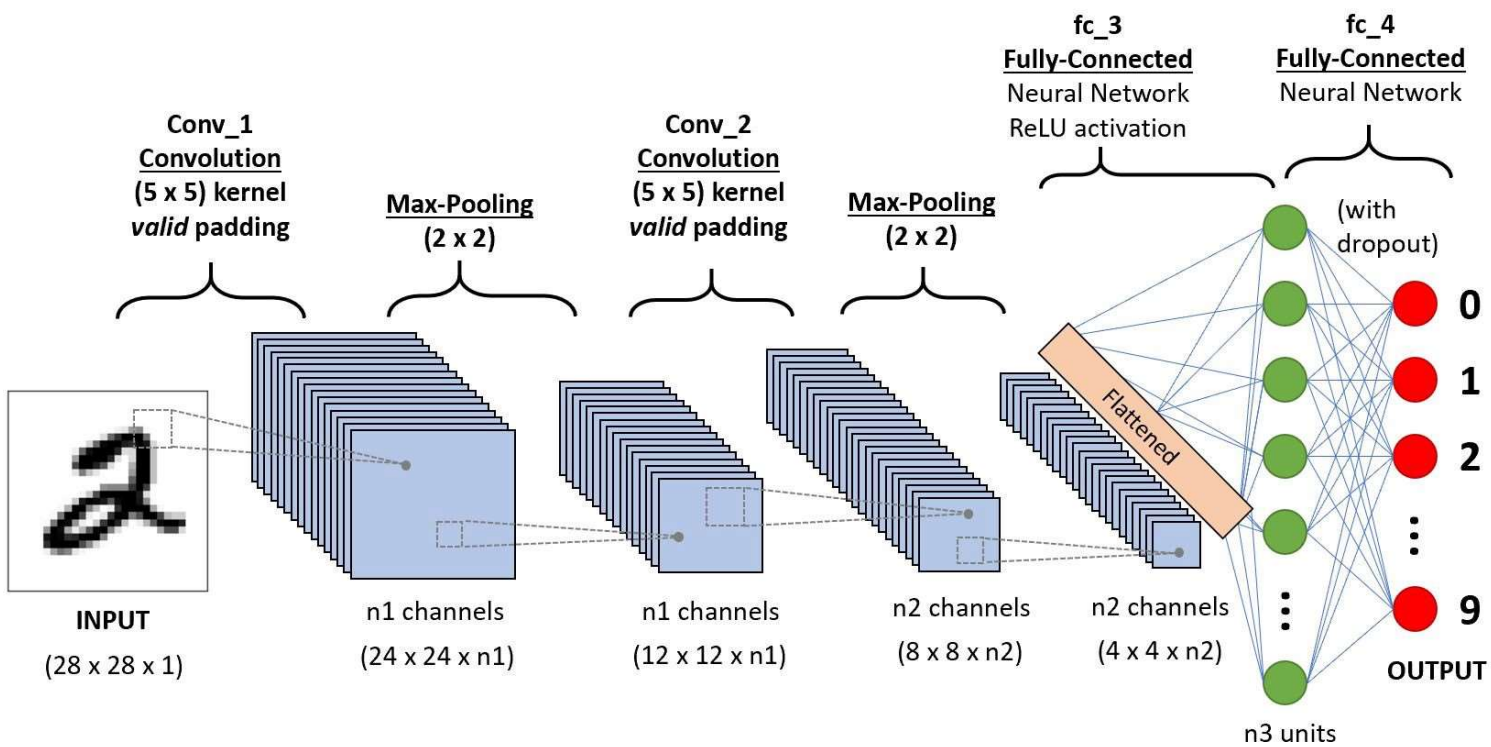
So far we have learned about fully-connected neural networks, like below:



Does that make sense for (two-dimensional) images?

Source: Lamarr

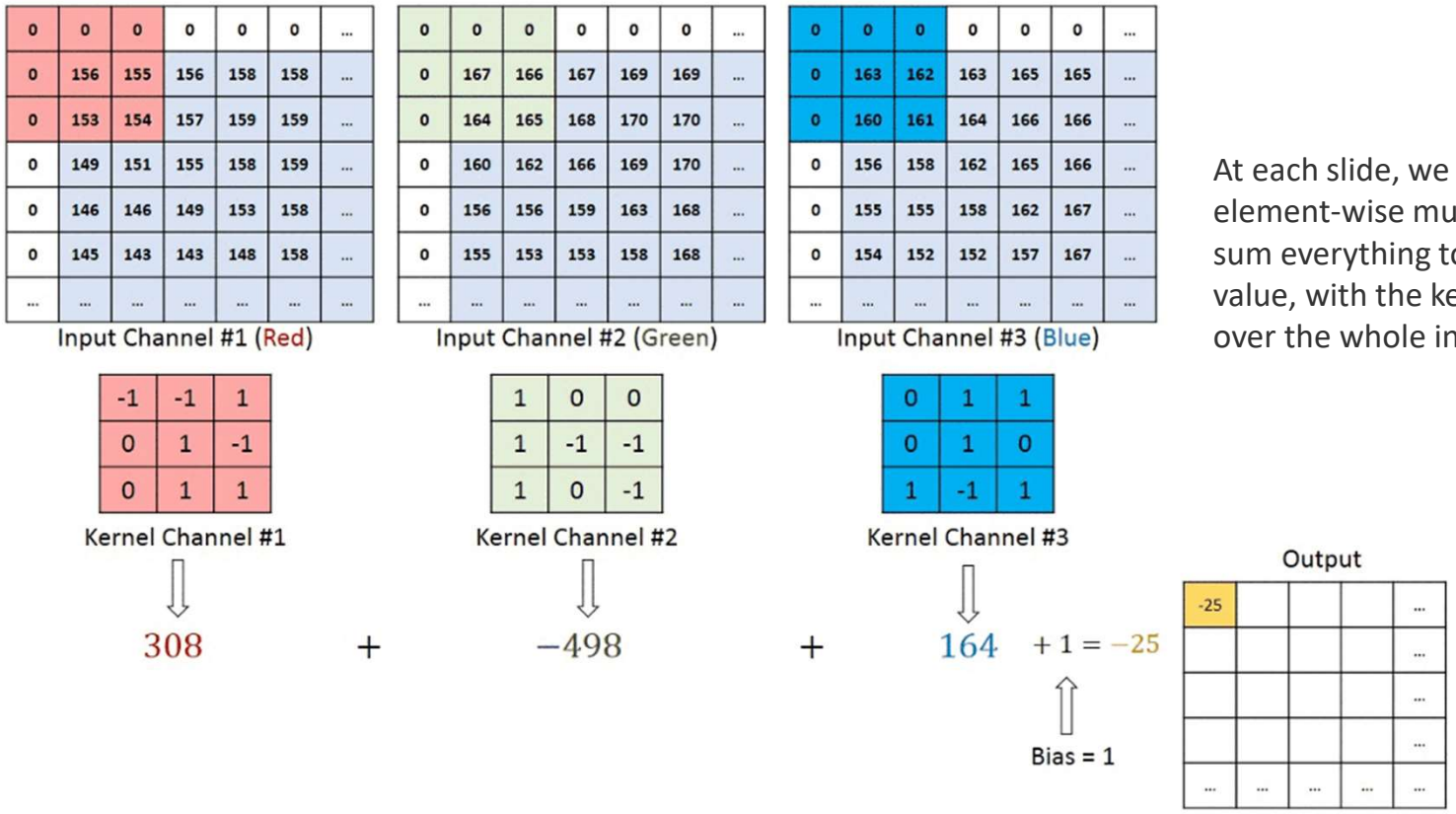
Convolutional Neural Networks (CNN)



Source: Jonas Bokstaller

CUNY School of Professional Studies

Convolutions



At each slide, we perform an element-wise multiplication and sum everything to get a single value, with the kernel is sliding over the whole input.

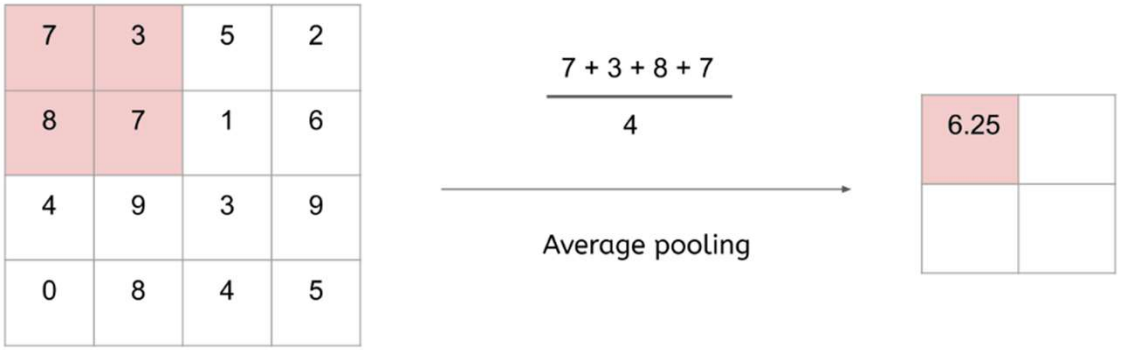
Source: https://medium.datadriveninvestor.com/convolutional-neural-networks-3b241a5da51e

CUNY School of Professional Studies

Pooling

Max Pooling

Average pooling



Source: https://www.geeksforgeeks.org/cnn-introduction-to-pooling-layer/

The goal of pooling is to reduce the height and width of our image but not the number of channels by using a stride > 1

CUNY School of Professional Studies

Convolutions

Filters capture spatial/temporal dependencies
Reduces dimensionality faster

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

6		

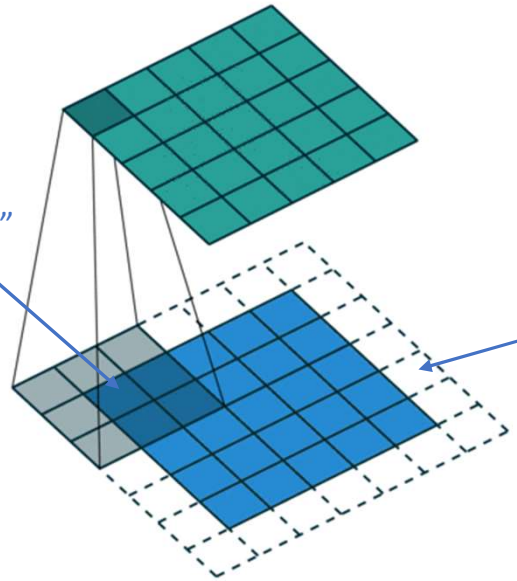
7x1+4x1+3x1+
2x0+5x0+3x0+
3x-1+3x-1+2x-1
= 6

Some examples of kernels

Original	*Gaussian Blur*	*Sharpen*	*Edge Detection*
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$

Source: https://medium.com/analytics-vidhya/understanding-convolution-operations-in-cnn-1914045816d4

Padding and stride

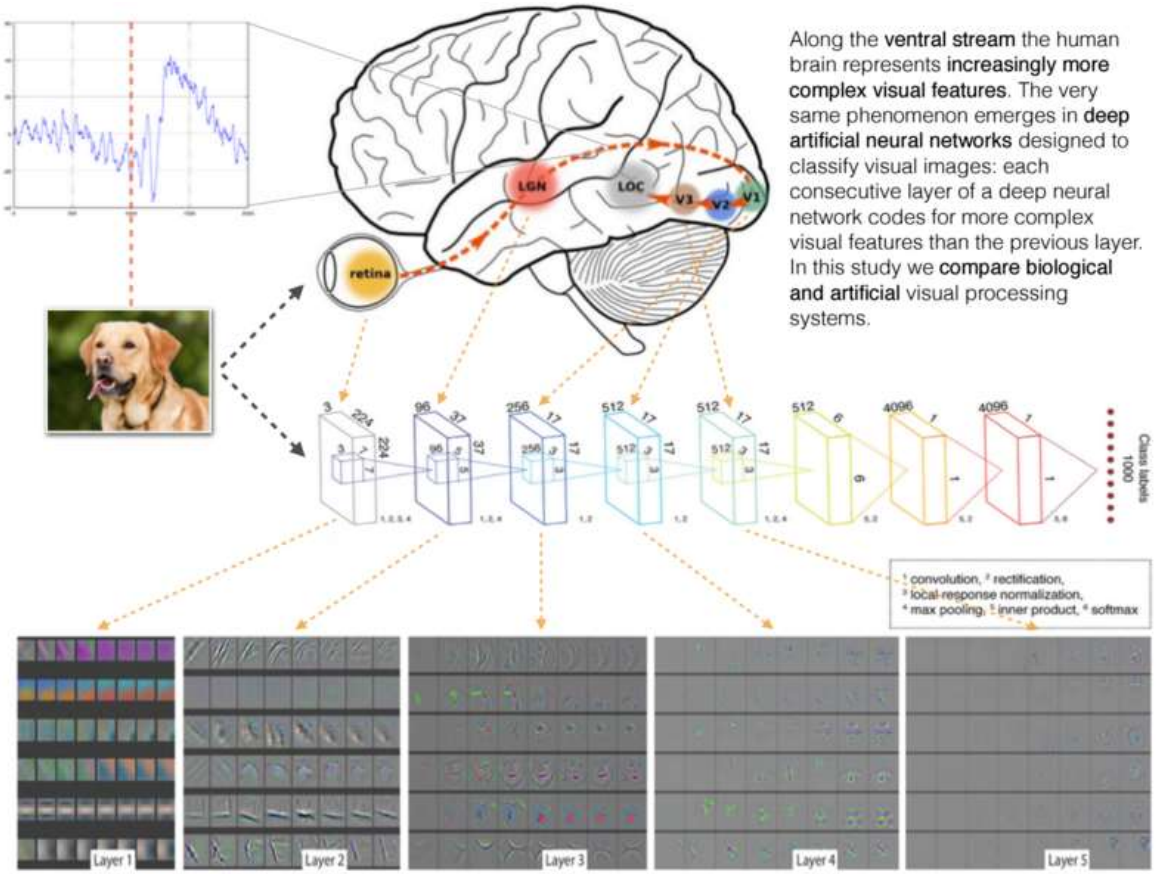


Stride is the “steps” between kernel convolutions

Padding indicates outer rows and columns (set to zero)

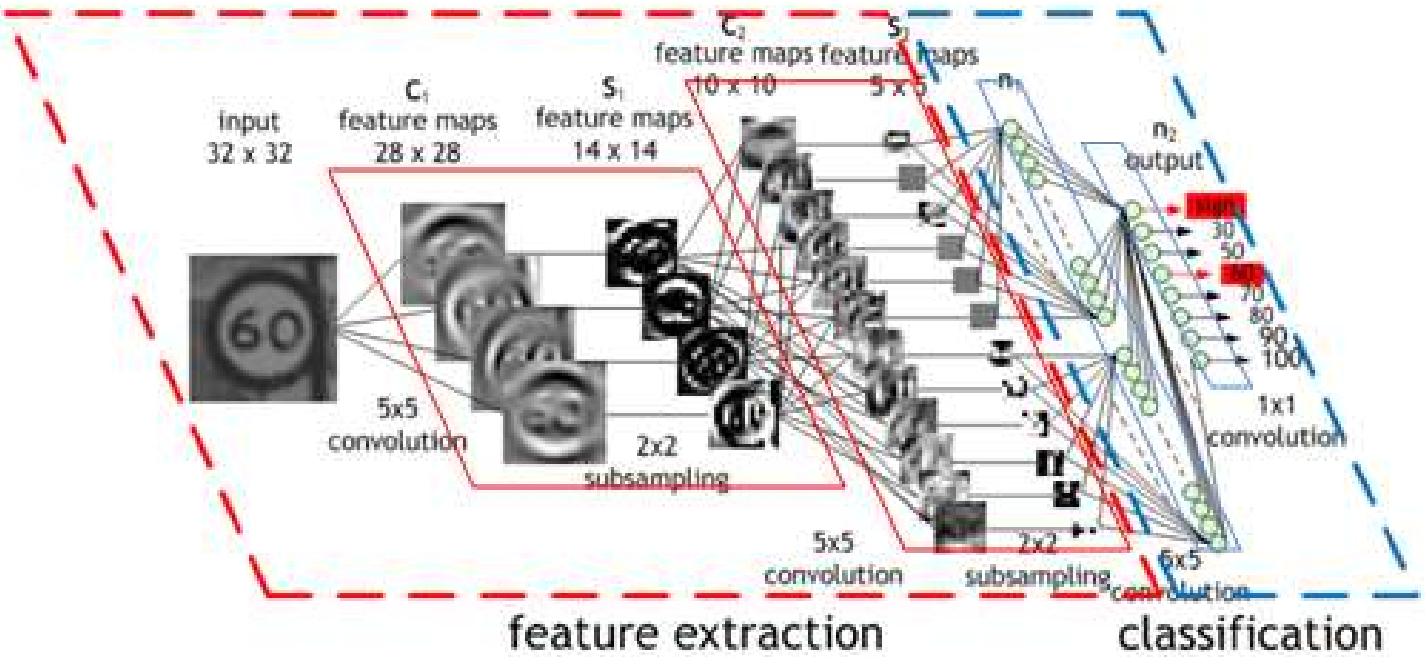
Source: https://medium.datadriveninvestor.com/convolutional-neural-networks-3b241a5da51e

The Brain's visual cortex



CUNY School of Professional Studies

CNN example



CUNY School of Professional Studies